same data. Links are usually created with the ln command; to create the *file2* hard link to *file1*, the command would be ln *file1 file2*. After this, let us list the directory contents, with inodes, as shown below:

```
bash-3.00$ ls -li
total 2
2889771 -rw-r--r-- 2 user1 grp1 5 Apr 27 2012 file1
2889771 -rw-r--r-- 2 user1 grp1 5 Apr 27 2012 file2
```

You can see that both files have the same inode number 2889771. The third column shows the number of hard links—in the above case, it's two for both, since the one inode has two directory entries referencing it.

To notice something different, create a sub-directory, *dir1*. Do a listing again; you'll see that the link count of the directory is 2, and not 1, as for a regular file:

```
2889767 drwxr-xr-x  2 user1 grp1 512 Apr 26  2012 dir1
```

Why does even an 'empty' new directory have a link count of 2? Because of the following reasons:

- Its parent directory contains a directory entry for the name-to-inode mapping—that's one reference.
- In the directory itself, there is always created a . (dot) entry signifying the current directory, referencing the same inode for the directory—that's the second.

Now, there's also a default .. (dot-dot, parent directory) entry created, and so obviously the parent directory's link count also increases by 1.

Now let's look at *soft links*, also called *symbolic links*. These are actually files containing the path to a target (linked file) and are created by adding the -s parameter to the *ln* command. For example:

```
bash-3.00$ ln -s file1 file3
bash-3.00$ ls -ltri
total 3
2889771 -rw-r--r--  2 user1 grp1 5 Apr 27  2012 file2
2889771 -rw-r--r--  2 user1 grp1 5 Apr 27  2012 file1
2889769 lrwxrwxrwx  1 user1 grp1  5 Apr 27  2012 file3 -> file1
```

Here, *file3* is a soft link to *file1*; it's a separate file, with its own inode number. Note that if we try to open *file3* with an editor, *file1's* content is what we'll actually see—most programs follow symbolic links to get at the target files. To view the actual contents of the link file, not the target, use a command like *readlink*, as follows:

```
bash-3.00$ readlink file3
file1
```

Symbolic links are more flexible than hard links: you cannot create a hard link on one file-system to a file on another file-system (say, a different HDD partition mounted by the OS). You can, however, create symbolic links across file-systems.

Accessing file attributes with C

Here is an example of accessing file/directory attributes, which is called *my_dirtraverse.c*:

```
#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<dirent.h>
#include<sys/stat.h>
#include<sys/types.h>
void traversedir (char *);
int main () {
        char cdir[1024];
        /*get the path of current working directory using getcwd*/
        getcwd(cdir,1024);
        traversedir(cdir);
        return 0;
}
void traversedir( char *dir) {
        struct dirent *dp; /*dp is a pointer to a
        structure representing directory Entry*/
        struct stat sbuf; /*This a structure, which contains the
attributes
        of a file or directory*/
        char tempdpath [1024];
DIR *dfd; /* DIR - Assume it is a structure which contains the file
descriptor for a open directory and the directory entry*/
        printf("Directory %s\n",dir);
        /*Trying to open a directory using opendir function, which
returns
        a pointer to DIR structure*/
        if ((dfd=opendir (dir))==NULL) {
                printf ("Cannot open %s",dir);
                return;
        }
        /*In a while Loop read the Directory Entries using
readdir*/
        while ((dp=readdir (dfd))!=NULL) {
                /*every Directory is having the dot & dot dot
entries*/
                if (strcmp (dp->d_name,".")==0) {
                        continue;
                }
                else if (strcmp(dp->d_name,"..")==0) {
                        continue;
                }
                else {
                        /*checking whether it a directory*/
```